

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1558

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 40

Duration :60 Minutes

Code of Conduct

I Santhosh kumar.M (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Santhosh

Assessment Tasks

Industry Problem Solving Task

- Retail Data Optimization Problem** - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.
- Social Media Fraud Detection Problem** - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.
- Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.
- Banking Fraud Detection Problem**
A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

5. **Agriculture Smart Farming Problem**

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded:

Q1:Industry Problem Solving Tasks – Big Data Architecture

1. Retail Data Optimization Problem

Problem

A retail chain operates physical stores and an online platform. It generates large amounts of sales, inventory, customer, and browsing data. The company needs:

- Real-time inventory tracking
- Demand forecasting
- Personalized marketing recommendations

Proposed Big Data Architecture

Data Sources → Data Ingestion → Data Storage → Stream/Batch Processing → Analytics & ML → Applications

1. Data Sources

Data can be collected from:

- Point-of-sale (POS) systems in physical stores
- E-commerce transactions
- Inventory and warehouse management systems
- Customer loyalty programs
- Website and mobile-app clickstreams
- Product searches and browsing history
- Customer reviews
- Social media interactions
- Supplier data

2. Data Ingestion

Apache Kafka can be used to collect real-time events such as purchases, inventory updates, and website clicks.

For batch data, tools such as **Apache NiFi**, ETL tools, or cloud ingestion services can be used. Kafka provides high-throughput, fault-tolerant streaming and allows multiple downstream applications to consume the same events.

3. Data Storage

A **data lake** can store structured, semi-structured, and unstructured data.

Possible technologies include:

- Hadoop HDFS
- Amazon S3
- Azure Data Lake Storage
- Google Cloud Storage

A distributed database such as **Apache Cassandra** can be used when very fast access to large volumes of operational data is required.

A data warehouse such as **Snowflake**, **BigQuery**, or **Amazon Redshift** can support business intelligence and reporting.

4. Data Processing

Apache Spark can process both historical and streaming data.

For real-time processing:

- Spark Structured Streaming
- Apache Flink

can process inventory and customer events with low latency.

For batch processing, Spark can analyze historical sales and inventory data.

5. Demand Forecasting

Machine learning models can predict future demand based on:

- Historical sales
- Seasonality
- Holidays
- Discounts
- Weather
- Location
- Product popularity
- Customer behavior

Suitable algorithms include:

- ARIMA/SARIMA
- Random Forest
- XGBoost
- LSTM neural networks

The output can help the retailer decide **how much stock to maintain and where to distribute it**.

6. Personalized Marketing

A recommendation engine can analyze customer behavior and recommend products.

Possible approaches include:

- Collaborative filtering
- Content-based filtering
- Association-rule mining
- Hybrid recommendation systems

For example, if customers frequently purchase a mobile phone with a particular case, the system can recommend that case to customers purchasing the same phone.

7. Visualization

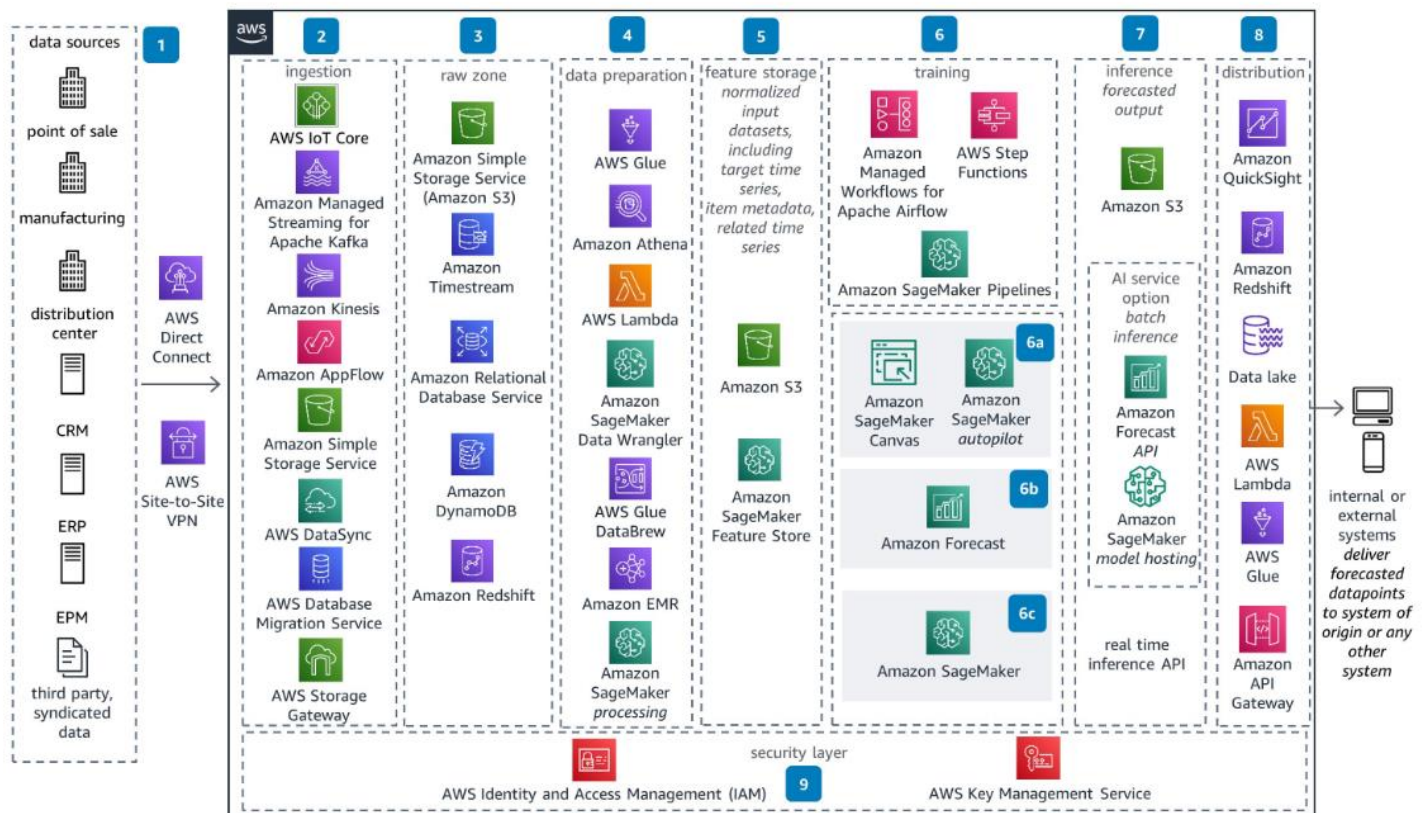
Tools such as **Power BI**, **Tableau**, or **Looker** can provide dashboards showing:

- Current inventory
- Sales trends
- Forecasted demand
- Customer segments
- Product performance

Result

The architecture enables **real-time inventory visibility, accurate demand forecasting,**

reduced stock-outs, lower excess inventory, and personalized customer recommendations



2. Social Media Fraud Detection Problem

Problem

A social media platform processes billions of posts, comments, likes, and shares every day. It needs to identify:

- Fake accounts
- Spam
- Coordinated fraudulent behavior
- Misinformation
- Suspicious user activity

The solution must work in real time or near real time.

Proposed Architecture

Social Media Events → Kafka → Stream Processing → Feature Extraction → ML Models → Risk Scoring → Action/Alert

1. Data Sources

The platform collects:

- Posts

- Comments
- Likes
- Shares
- Friend/follower relationships
- Login information
- IP/device information
- Account creation information
- User activity patterns
- Reports and moderation decisions

2. Real-Time Data Ingestion

Apache Kafka can act as the central event-streaming platform.

Every interaction can become an event:

User → Like → Post → Timestamp → Device → Location

Kafka can handle extremely high event volumes and distribute events to multiple processing systems.

3. Stream Processing

Apache Flink or **Spark Structured Streaming** can process events continuously.

For example, the system can identify:

- Hundreds of likes within a few seconds
- Thousands of accounts behaving identically
- Sudden coordinated sharing
- Multiple accounts using the same suspicious device patterns

Flink is particularly useful when very low latency and stateful stream processing are required.

3. Feature Engineering

The system can calculate features such as:

- Account age
- Number of posts per hour
- Followers/following ratio
- IP/device reuse
- Similarity between posts
- Abnormal login patterns
- Sharing frequency
- Network connections
- Sentiment and language patterns

4. Machine Learning

Several ML models can be used.

Fake-account detection:

- Random Forest
- XGBoost
- Logistic Regression
- Neural networks

Misinformation detection:

- Natural Language Processing (NLP)
- Transformer-based language models
- Text classification
- Source credibility analysis

Network-based detection:

- Graph algorithms
- Graph Neural Networks (GNNs)
- Community detection

5. Real-Time Risk Scoring

Each event or account can receive a risk score.

For example:

- **0–30:** Low risk
- **31–70:** Medium risk
- **71–100:** High risk

High-risk activity can trigger:

- Additional verification
- Reduced content distribution
- Temporary account restrictions
- Human moderator review

6. Storage

Historical data can be stored in:

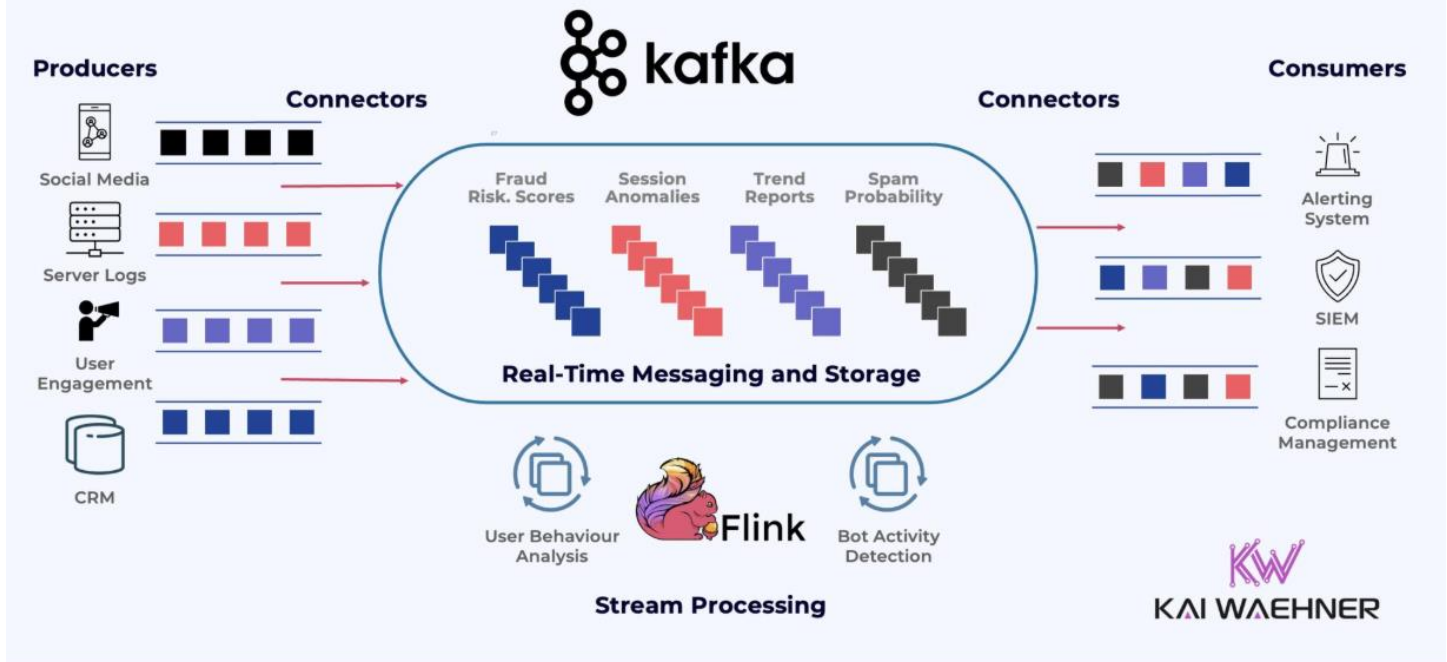
- Data lake
- HDFS
- Cloud object storage
- NoSQL databases

Historical information is useful for training and improving ML models.

Result

The combination of **Kafka + Flink/Spark + machine learning + graph analytics** provides scalable and near-real-time fraud detection while allowing the platform to process billions of interactions.

Event-driven Architecture with Data Streaming for Spam, Abuse and Fraud Detection



3. Government Data Platform Problem

Problem

A government agency needs to integrate:

- Census data
- Economic data
- Geospatial data
- Departmental datasets

Different government departments must be able to share data while maintaining **interoperability, privacy, security, and governance**.

Proposed Architecture

Government Departments → Data Ingestion → Data Lake → Data Integration → Governance Layer → Secure Data Services → Departments/Applications

1. Data Sources

Examples include:

- Census databases
- Tax systems
- Economic statistics
- Healthcare statistics
- Transportation systems
- Land records
- Satellite/geospatial datasets
- Education databases

2. Data Integration

Data should be integrated using common standards and APIs.

Technologies can include:

- Apache Kafka
- Apache NiFi
- REST APIs
- ETL/ELT pipelines

Standard data formats such as **JSON, XML, CSV, and Parquet** can improve interoperability. For geospatial data, standards from organizations such as the **Open Geospatial Consortium (OGC)** can be followed.

3. Data Lake

A government-wide data lake can store large volumes of data.

Data can be divided into:

- Raw zone
- Processed/cleaned zone
- Curated/analytics zone

Object storage or Hadoop-based storage can be used.

4. Metadata Management

A central **data catalog** should record:

- Dataset name
- Data owner
- Data source
- Data format
- Data quality
- Update frequency
- Access classification

This makes it easier for departments to discover and understand datasets .

6. Data Governance

A strong governance framework should define:

- Data ownership
- Data stewardship
- Data quality standards
- Data retention policies
- Data classification
- Data sharing rules
- Audit requirements

A **Master Data Management (MDM)** approach can ensure that important entities such as citizens, locations, organizations, and government departments are represented consistently.

6. Security and Privacy

Because government datasets may contain sensitive information, the platform should use:

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)
- Encryption at rest

- Encryption in transit
- Multi-factor authentication
- Data masking
- Anonymization/pseudonymization
- Audit logging

Only authorized users should access sensitive datasets.

7. Data Quality

Automated validation can detect:

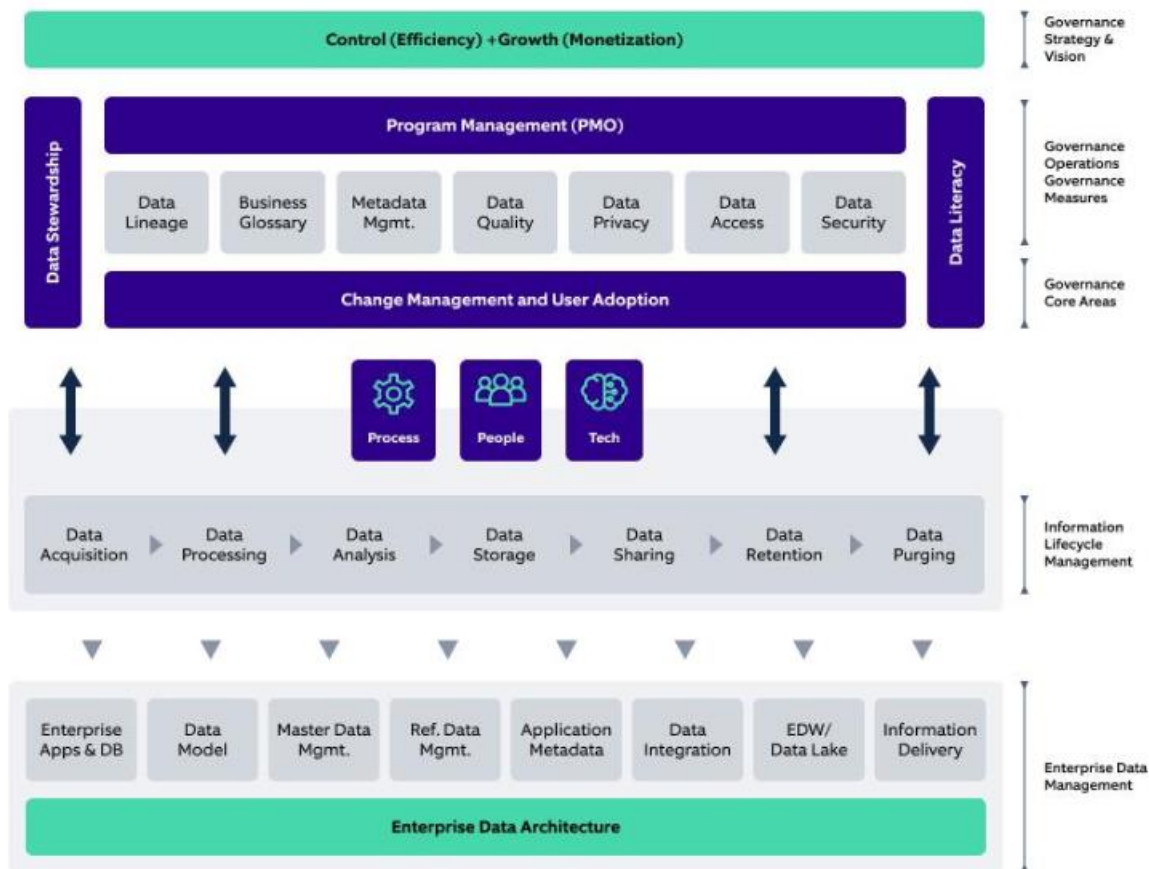
- Missing values
- Duplicate records
- Incorrect formats
- Inconsistent identifiers
- Outdated information

Data-quality rules should be applied before datasets are made available to other departments.

Result

A centralized but governed government data platform provides **interoperability, secure data sharing, better data quality, reduced duplication, and improved decision-making** across departments.

Data Governance Framework



4. Banking Fraud Detection Problem

Problem

A bank processes millions of transactions every minute and must detect fraudulent transactions with:

- Very low latency
- High accuracy
- High scalability
- Continuous analysis of historical and streaming data

Proposed Architecture

Transaction → Kafka → Stream Processing → Feature Engineering → ML Fraud Model → Risk Score → Decision Engine → Alert/Block

Historical data is processed separately to train and improve the models.

1. Data Sources

The system can use:

- Credit/debit card transactions
- ATM transactions

- Online banking transactions
- Mobile banking transactions
- Account information
- Customer transaction history
- Device information
- Location information
- Previous fraud cases

2. Real-Time Data Ingestion

Apache Kafka can receive transaction events at very high speed.

Example:

Transaction ID

Customer ID

Amount

Merchant

Time

Location

Device

Payment method

Kafka provides fault tolerance and allows multiple services to process transaction streams simultaneously.

3. Stream Processing

Apache Flink or **Spark Structured Streaming** can process transactions in real time.

The system can calculate features such as:

- Number of transactions in the last 5 minutes
- Average transaction amount
- Distance between consecutive transactions
- Number of failed transactions
- Unusual merchant category
- Device changes
- Unusual spending pattern

4. Machine Learning

Several algorithms can be combined.

Supervised learning:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks

These models learn from previously labeled fraudulent and legitimate transactions.

Unsupervised learning:

- Isolation Forest
- Autoencoders
- Clustering

These methods can identify previously unknown patterns.

5. Rule-Based Detection

ML can be combined with business rules.

For example:

If a card is used in two geographically distant locations within an impossible travel time, increase the fraud score.

Rules are useful because banks often need deterministic controls in addition to ML predictions.

Fraud Scoring

Each transaction receives a probability/risk score.

For example:

Transaction



Feature extraction



ML model



Fraud probability = 0.94



High-risk transaction



Block / verify / alert

Low-risk transactions can proceed normally, while high-risk transactions can require additional authentication or manual review.

7. Historical Data

A data lake can store historical transactions.

Apache Spark can be used to:

- Train models
- Analyze customer behavior
- Identify fraud trends
- Generate features
- Retrain models periodically

8. Model Monitoring

The bank should continuously monitor:

- False positives
- False negatives
- Model accuracy
- Model drift
- Fraud detection rate
- Processing latency

Result

A **Kafka + Flink/Spark + ML + rule engine + data lake** architecture can provide real-time fraud detection while maintaining scalability and high detection accuracy.



5. Agriculture Smart Farming Problem

Problem

Smart farming systems generate data from:

- Soil sensors
- Weather stations
- Drones
- Satellites
- GPS devices
- Irrigation systems
- Agricultural machinery

Farmers need to improve **crop yield, irrigation, fertilizer usage, and resource efficiency.**

Proposed Architecture

Sensors/Drones/Satellites → IoT Gateway → Kafka/MQTT → Data Lake →

Stream/Batch Processing → ML/Analytics → Farmer Dashboard/Automated Actions

1. Data Collection

IoT sensors can collect:

- Soil moisture

- Soil temperature
- Soil pH
- Nutrient levels
- Air temperature
- Humidity
- Rainfall
- Crop health

Drones can capture images of agricultural fields.

Satellite imagery can provide large-scale information about vegetation and crop conditions.

2. Data Ingestion

Protocols such as **MQTT** can efficiently transmit IoT sensor data.

Apache Kafka can be used when large-scale event streaming and integration with other systems are required.

2. Data Storage

A cloud data lake can store:

- Sensor data
- Drone images
- Satellite images
- Weather data
- Historical crop yields
- Farm management records

Technologies can include:

- HDFS
- Amazon S3
- Azure Data Lake
- Google Cloud Storage

3. Real-Time Processing

Apache Spark Streaming or **Apache Flink** can process incoming sensor data.

For example:

Soil moisture < threshold



Streaming system



Irrigation recommendation



Automatic irrigation

This reduces unnecessary water consumption.

4. Predictive Analytics

Machine learning can predict:

- Crop yield
- Irrigation requirements
- Pest outbreaks

- Disease risk
- Fertilizer requirements
- Harvest time

Algorithms can include:

- Random Forest
- XGBoost
- Regression models
- Time-series forecasting
- Neural networks

5. Image Analytics

Drone and satellite images can be analyzed using **computer vision and deep learning**.

For example, image classification can identify:

- Diseased plants
- Weed growth
- Crop stress
- Areas with insufficient irrigation

CNN-based models can be used for image analysis.

6. Weather-Based Analytics

Weather data can be combined with crop and soil information.

For example:

If heavy rainfall is predicted and soil moisture is already high, irrigation can be reduced or stopped.

This helps farmers make better decisions.

7. Farmer Dashboard

A mobile or web dashboard can display:

- Soil moisture
- Weather forecasts
- Crop health
- Irrigation recommendations
- Fertilizer recommendations
- Pest alerts
- Expected crop yield

Result

The solution enables **precision agriculture**, where water, fertilizer, pesticides, and other resources are applied only where and when they are needed. This can increase crop productivity while reducing costs and environmental impact.

